Shortening the K215 Correctness Proofs

One implication core and semantic instruction contracts

Research and implementation proposal for ProveIt

21 October 2026

Abstract

The most useful immediate shortening is to share two implicational derivations that remain separate in the current proof. The maintained 58-step dominated implication core already contains most of the unrestricted basis used elsewhere. Five additional modus-ponens steps derive its missing S theorem. Instantiating the shared core with the always-true guard then replaces a separate 107-helper derivation in `Lukasiewicz.lean`. The two proof files shrink from 819 to 435 lines and from 42,715 to 19,674 bytes. The combined derivations use 101 fewer modus-ponens steps.

A complementary change makes register-machine simulation consume semantic instruction contracts. The existing Boolean site checker becomes one provider of those contracts. This makes separately proved decoder, primitive and return contracts usable without proving completeness of a fixed-fuel checker. An isolated Lean implementation preserves all four legacy simulation interfaces and proves the same halting equivalence through the semantic interface.

The new implication proofs and adapters were checked with Lean 4.32.0 and have no axiom dependencies. The semantic refactor, its 13-module project closure and typed compatibility clients were checked as well. These are focused checks of proposed changes, not a fresh build of the complete K215 endpoint. The packet contains patches, exact replacements, reproducible verification drivers, raw logs and a clearly labelled, uncompiled Rocq interface draft.

Result	Status in this delivery
Shared implication proof	63-step core and exact generic adapters kernel-checked
Semantic simulation interface	Replacement module and old/new clients kernel-checked
Diagonal-guard obstruction	Three-element countermodel, with Lean and Python checks
Full K215 endpoint after patches	Integration and complete downstream rebuild remain
Rocq interface implementation	Design draft only; no new Rocq compilation claimed
Machine and source	Same K215 target; this work changes proof infrastructure

Contents

1 Snapshot, scope and remaining duplication 3

2 Immediate shortening: share the implication proofs 4

3 The five-step extension, with explicit proof 5

4 Semantic instruction contracts 6

5 Positive progress and exact continuation context 8

6 Gradual migration to shared primitive proofs 9

7 The Rocq route 10

8 An obstruction to diagonal-only simplification 12

9 Validation and limits 12

10 Recommended integration order 14

A Source and evidence map 14

1 Snapshot, scope and remaining duplication

1.1 A fixed baseline

The reviewed snapshot is ProveIt commit

`2ca3a5c16f0876d2cab842056353c03a1ed2f5f0`

All source comparisons and patches use that commit. A subsequent check found `6f0d42d36850`; its only change was an incoming ZIP under `docs/incoming`. The proof sources used here were unchanged.

The maintained machine has 215 working states and 430 actions. Its dense-table SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`

and its source JSON SHA-256 is

`17af207195179b2508bf91154cd00739374f8862207ca21b9a3cb9098aa6f5ba`

Both were recomputed locally and agree with the repository verification record [1]. Neither artifact is modified by these patches.

The established endpoint is the external proof-assistant theorem

$$\mathrm{Halt}(K215, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Refutable}(\mathrm{ZFC}),$$

where refutability means the existence of a finite derivation of falsity. Negation gives the nonhalting/syntactic-consistency equivalence. The repository explicitly distinguishes this from an internal PA proof or a derivation encoded in ZFC's own proof syntax [1]. The user's intended internal ZFC equivalence remains the metatheoretic target; shortening the external correctness development does not itself close that separate formalization task.

1.2 What the recent commits already accomplished

Several substantial improvements are already present and should be credited:

- Lean's dominated implication argument is a representation-independent 58-step condensed-detachment derivation. It uses reflexivity and growth into either implication operand; it already needs no transitivity, formula decoder, injectivity or object-language truth constant.
- Lean's source correspondence uses 1,426 normalized certificate entries, balanced storage and one-sided checking.
- The seed directions share bundle semantics.
- The Home return scanner has a shared proof. Phase nine retains all 4,751 leaves and 6,176 branch outcomes and reports 984.608 seconds for nine fresh site batches. Its comparison with the historical 1,649.340 seconds is historical, not a controlled paired benchmark.
- The decoder has a checked, exact source-bound prefix theorem. It reaches a primitive entry port while retaining the whole consumed word. Primitive factoring remains open.

These statements are grounded in the maintained sources and status records [3, 2, 7, 5].

The production Rocq tree is at a different implementation boundary. Its `DomRho8.v` retains the older checked certificate replay; the newer 52-step rendering is a proposal. Its live `Source215.v` still invokes the signed offset checker in both directions. A normalized Rocq implementation exists in a separate proposal package [5, 10, 11]. Intermediate commit messages alone do not identify the current maintained implementation.

1.3 Three meanings of shortening

A proof improvement can remove mathematical duplication, reduce the source and dependency surface, or reduce kernel computation and rebuild time. These are different measurements. The shared implication change demonstrates the first two. No endpoint timing gain is claimed. The semantic interface enables future improvements but initially adds code. Neither source-line counts nor symbolic operation counts should be presented as a wall-clock measurement.

2 Immediate shortening: share the implication proofs

2.1 The duplication

At the baseline, `DominatedImplication.lean` contains 58 numbered MP lemmas. Separately, `Lukasiewicz.lean` contains 107 private `cd_` helpers: one axiom leaf and 106 MP steps. The latter derives K, Peirce, composition and S from unrestricted rho8 [3, 4].

The shared core already proves K at `d12`, Peirce at `d18`, and composition at `d58`. Its public exports are rho8, composition and prefixing. Thus most of the second derivation is available inside the first. Its missing public S theorem requires only five additional steps.

This targets duplication at a mathematical boundary. The surviving proof keeps short, named nodes; there is no wholesale inlining of large substitution terms. The separation between an abstract implication argument and its numerical interpretation is retained.

2.2 The abstract interface

Let A be an arbitrary type, $a \Rightarrow b$ an arbitrary binary operation, and $T \subseteq A$ the designated predicate. Write

$$R(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

The core assumes a relation $\preccurlyeq$ with

$$a \preccurlyeq a, \qquad u \preccurlyeq x \Longrightarrow u \preccurlyeq (x \Rightarrow y), \qquad u \preccurlyeq y \Longrightarrow u \preccurlyeq (x \Rightarrow y),$$

and inference data

$$T(a \Rightarrow b) \wedge T(a) \Longrightarrow T(b), \qquad u \preccurlyeq a \Longrightarrow T(R(a, b, c, u)).$$

The operation need not be injective, and elements of A need not decode as syntax. This is useful for numerical code systems that admit junk codes.

To recover `Lukasiewicz.Implicational`, take

$$u \preccurlyeq a \quad :\Longleftrightarrow \quad \top.$$

The unrestricted rho8 field supplies the restricted field by ignoring its trivial guard. The connective and predicate remain unchanged. Here $\top$ is a proposition in the metalanguage, not an object-language truth constant.

The essential adapter is:

```
private def fullLanguage (imp) where
  imp := imp
  below := fun _ _ => True
  refl := fun _ => True.intro
  grow_left := fun _ _ => True.intro
  grow_right := fun _ _ => True.intro
```

The packet contains the complete typed version. The original `Implicational` structure and four public theorem types are preserved.

2.3 Exact source reduction

Proof source	Before	After	Line change
`DominatedImplication.lean`	295	330	+35
`Lukasiewicz.lean`	524	105	−419
Combined	819	435	−384

The combined byte count changes from 42,715 to 19,674: 23,041 bytes removed, approximately 53.9%. The line reduction is approximately 46.9%. These measurements concern the two deployed proof sources, not the archive, which also includes patches and verification copies.

The renderer adds 21 lines and derivation data adds 10. Including these producers, the four-file change removes 353 lines and 21,813 bytes. The old developments contain $58 + 106 = 164$ MP steps, versus 63 in the shared core: 101 explicit MP steps eliminated. This does not assert minimality of 63 or an analogous reduction in compiled objects or endpoint time.

The Tarski section after the generic `Lukasiewicz` namespace is byte-identical: 1,358 bytes. Its ex-falso argument continues to use rho6. The refactor does not claim that rho8 alone proves arbitrary falsity elimination.

3 The five-step extension, with explicit proof

Write $a \to b$ for the abstract connective. Every schema below is designated under T; MP uses the supplied inference field, without a semantic interpretation of the arrows. The old core supplies

$$A(a,b) = a \to ((a \to b) \to b), \qquad \texttt{d36},$$
$$B(a,b,c) = (a \to b) \to ((b \to c) \to (a \to c)), \qquad \texttt{d58},$$
$$D(a,b,c) = (a \to (b \to c)) \to ((b \to a) \to (b \to c)), \qquad \texttt{d47}.$$

Lemma 3.1. *The following five schemata follow by one MP step each in the displayed order:*

$$Q(a,b,c) = (((a \to b) \to b) \to c) \to (a \to c), \tag{1}$$
$$E(a,b,c,d) = (((a \to b) \to (c \to b)) \to d) \to ((c \to a) \to d), \tag{2}$$
$$V(a,b,c) = (a \to b) \to ((a \to (b \to c)) \to (a \to c)), \tag{3}$$
$$C(a,b,c) = (a \to (b \to c)) \to (b \to (a \to c)), \tag{4}$$
$$S(a,b,c) = (a \to (b \to c)) \to ((a \to b) \to (a \to c)). \tag{5}$$

Proof. For (1), apply MP to $B(a, (a \to b) \to b, c)$ and $A(a, b)$. For (2), apply MP to

$$B(c \to a, (a \to b) \to (c \to b), d) \quad \text{and} \quad B(c, a, b).$$

For (3), use

$$E(b, c, a, (a \to (b \to c)) \to (a \to c)) \quad \text{and} \quad D(b \to c, a, c).$$

For (4), use

$$E(b \to c, c, a, b \to (a \to c)) \quad \text{and} \quad Q(b, c, a \to c).$$

Finally use

$$C(a \to b, a \to (b \to c), a \to c) \quad \text{and} \quad V(a, b, c).$$

Each first premise has the second as its antecedent after the displayed substitution. The generated proof terms give the substitutions explicitly. □

These become `d59` through `d63`. K and Peirce are exported from their existing nodes. The rho8, composition and prefixing exports keep their statements.

Theorem 3.2 (Shared unrestricted basis)**.** *For every type A, binary operation $\to$ and predicate T, closure under MP and all unrestricted rho8 instances imply K, Peirce, composition and S. The implementation obtains all four from the same 63-step dominated core via the always-true relation adapter.*

This theorem is mathematically familiar. The concrete result is the sharing refactor against this repository snapshot and the five-step extension that makes it economical. A standalone Lean file contains the entire generated core followed by the exact replacement generic adapter section. It imports only `Init`. All six core exports and all four adapter theorems were kernel-checked and report no axioms.

The producer is synchronized: regenerating the core gives the proposed file byte for byte. The non-core Nat Lean and Rocq renderings are compared with the baseline renderer on their original derivation. Their existing output is unchanged.

4 Semantic instruction contracts

4.1 Why the Boolean boundary obstructs sharing

The maintained `KSim.Covered` requires every decoder walk to produce an edge with `checkEdge = true`. The `runTo` and `runHalt` checks invoke a symbolic executor with `FUEL = 4000`. Simulation unfolds the Boolean tests and invokes their soundness lemmas instruction by instruction [6].

A semantic proof that a decoder and primitive tail reach the intended configuration does not imply that this fixed-fuel evaluator returns `true`. Soundness is not the converse completeness theorem. Thus a separately proved macro cannot simply replace an accepted Boolean result.

An earlier incoming proposal already identified this boundary [8]. This packet implements the semantic interface, its adapters and simulation theorem, and checks old and new clients. The conceptual proposal is credited; the checked implementation is the advance.

4.2 Separate specification from producer

Define SiteContract(M, E, Q, e) as the prescribed effect of edge e, with full consumed word Q, from symbolic entry port E. The entry is a parameter so a decoder prefix can be composed with a primitive and return proof. Define

$$\mathrm{SiteSpec}(M, Q, e) \quad :\Longleftrightarrow \quad |Q| \le 48 \ \wedge \ \mathrm{SiteContract}(M, \mathrm{start}(Q), Q, e).$$

The Lean implementation uses a structure with fields `length` and `effect`. It reuses the exact existing `tgtConf`, physical register map, growth, overflow and branch predicates.

For returning instructions, the contract gives positive reachability Reach^{+} for every natural gap n and every register list satisfying the old `Valid` predicate. There are two producers:

1. `checkEdge_sound` converts accepted old checks into semantic contracts, using the existing box-lemma hypothesis.
2. Shared mathematical proofs construct contracts directly, by composing decoder, primitive and continuation facts.

Only the first producer depends on the old evaluator.

The new coverage predicate states

$$\begin{aligned}\mathrm{SemanticCovered}(M, p) \ :\Longleftrightarrow \ \forall P \, \exists e, d, \ & \mathrm{walk}(p, p.\mathrm{nnodes} + 1, p.\mathrm{root}, P, 0) = \mathrm{some}(e, d) \\ & \wedge \mathrm{SiteSpec}(M, \mathrm{consume}(P, d), e).\end{aligned}$$

The stored path and consumed word are deliberately different: `consume` can include implicit zero padding.

4.3 Exact effects

Let q be the consumed word and $r^{*} = \mathrm{physOf}(r)$. All returning rows below assert positive reachability to the existing target.

Edge	Premise	Arguments to `tgtConf`
`inc r`	$r < 9$, `Valid none`	`incPath q`, `allOnes q`, increment r^*
`decnz r`, zero	$r < 9$, register $r^* = 0$	`breakPath 1 q`, `allOnes (stripPath false 1 q)`, no effect
`decnz r`, positive	$r < 9$, register $r^* \neq 0$	`incPath q`, `allOnes q`, decrement r^*
`brk k`	`Valid none`	`breakPath k q`, `allOnes (stripPath false k q)`, no effect
`cont k`	`Valid none`	`continuePath k q`, no overflow, no effect
`halt`	At least ten registers; state-230 terminal rows	Concrete halting from the entry configuration
`node`	Impossible as an instruction leaf	$\bot$

The halt case retains the terminal-row premise. The Boolean halt checker only recognizes state 230, so it cannot prove terminality for arbitrary tables. For returning branches, positivity is part of the specification.

4.4 One simulation, with compatibility wrappers

The new theorem gives

$$\text{RegisterPath.}\,\text{Halts}(p, c) \longleftrightarrow \text{KZip.}\,\text{Halts}(M, T)$$

from `SemanticCovered`, the old terminal-row premise and the unchanged relation `Rel p c T`. It takes no `BoxLemmas` premise. A producer may need box lemmas, but the simulation only needs the contract they establish.

The proof is the existing simulation with Boolean plumbing replaced by contract elimination. Register arithmetic, path cleanliness and full-tape reconstruction remain. The old public functions retain their types:

`sim_step`, `sim_stop`, `sim_halt`, `halts_iff`.

The step, halt and equivalence functions wrap semantic versions through `Covered.to_semantic`. The short stop proof remains independent because its old public interface never needed box lemmas.

The exact same typed legacy client compiles against original and replacement modules. The semantic audit exercises the new endpoint and composition rule. Matching legacy-client object hashes are supplementary evidence only, not equality of complete simulation proofs or endpoint artifacts.

5 Positive progress and exact continuation context

5.1 Why the reverse direction needs a positive clock

Suppose the represented machine configuration halts after t physical steps. If the register program can step, simulation yields $j + 1$ physical steps to the next related configuration. Determinism and absence of a successor from the terminal configuration imply $j + 1 \leq t$. The remaining run has length

$$t - (j + 1) < t,$$

so strong induction on actual machine time proves a finite register-program halting run.

If macros could simulate each source step with zero physical steps, this measure would not decrease. For example, a self-looping source configuration could be related to an already halted target, and reflexive reachability would vacuously simulate each source step. That would not preserve halting equivalence.

The decoder prefix may be empty. The verified composition rule is therefore

$$\mathrm{Reach}(M, s, u) \wedge \mathrm{Reach}^{+}(M, u, v) \Longrightarrow \mathrm{Reach}^{+}(M, s, v).$$

For halt, use $\mathrm{Reach}(M, s, u) \wedge \mathrm{Halt}(M, u) \Rightarrow \mathrm{Halt}(M, s)$. The new `SiteContract.prepend` proves these compositions for every edge.

5.2 Keep the full consumed word

For $Q = P +\!\!+ [b]$, the checked decoder reaches the zipper

$$\mathrm{zc}\Big(q, \mathrm{reverse}(P) +\!\!+ [\mathrm{true}], b :: 0^{\,n+(64-|Q|)} +\!\!+ (\mathrm{true} :: \mathrm{sepsR}(vs))\Big).$$

This is `leafPort pre bit q`. The consumed prefix is part of the continuation: return logic can revisit it, and break/continue depend on it.

Sharing only by physical state, read bit or short suffix would discard continuation information. Existing decoder research already finds a failure of unqualified state/read-bit sharing [9]. A valid primitive contract must retain the exact accepted role path and quantify over the whole prefix, gap, register list and branch. A short suffix can suffice inside a local decoder block with a proved frame theorem; that does not establish a complete primitive and return contract.

The useful quantifier order is

$$\exists \text{ source-determined port} \quad \forall n, vs \quad \exists \text{ run clock}.$$

Choosing a separate unrelated port for each tape instance is too weak for a finite shared interface. The maintained `entry_uniform` already provides the desired order [7].

5.3 Bounds and frame conditions remain mathematical obligations

The 48-bit bound and 64-cell `D0` parameter justify arithmetic in `rel_view`. The $r < 9$ hypothesis links logical registers to the physical map. Increment appends a spare block; positive decrement changes the growth term. Those effects are preserved exactly by retaining `tgtConf`.

Every gap value, including zero, is quantified. A symbolic true run may disappear at gap zero and join neighboring blank runs. The checked Home guard already handles that case. Neither the semantic refactor nor tests at a few sample gaps replace its universal argument.

6 Gradual migration to shared primitive proofs

The patch adds `semantic_covered_of_allSites`. Given a Boolean classifier `test` and a proof

$$\forall Q, e, \quad \texttt{test}(Q, e) = \mathrm{true} \Longrightarrow \mathrm{SiteSpec}(M, Q, e),$$

the existing structural `allSites` machinery supplies semantic coverage. This reuses tree totality and walk lemmas while allowing a different proof producer.

For a first primitive family, take a mixed provider:

$$\texttt{test}(Q, e) = \begin{cases} \text{true}, & \texttt{selected}(Q, e), \\ \texttt{checkEdge}(M, Q, e), & \text{otherwise.} \end{cases}$$

Its soundness requires an explicit theorem $\texttt{selected}(Q, e) \Rightarrow \text{SiteSpec}(M, Q, e)$; the fallback uses `checkEdge_sound`. The semantic audit typechecks this migration pattern.

A bounded first integration is:

1. Choose a primitive family with a manageable continuation invariant. The phase-nine packet identifies a first CB1 suffix proposal with 86 literal occurrences, explicitly source-only and uncompiled [2].
2. Prove its tail theorem from the exact `leafPort`, with the full role-path guard and old target configuration.
3. Compose the checked decoder prefix with the positive tail.
4. Route that family through the new provider, retaining old checks elsewhere.
5. Rebuild affected coverage and endpoint clients; benchmark old and mixed providers on the same table, source, compiler and machine.

The semantic module grows from 356 to 482 lines, an increase of 126. It removes none of the 6,176 outcomes in this delivery. That added interface is valuable because future shared proofs can now feed the endpoint without manufacturing old fixed-fuel Boolean results. The first complete primitive family should show a practical reduction before broadening the approach.

A decoder-role count or number of distinct first transitions is not a count of complete primitive contracts. This report does not assert that a few dozen proved runtime contracts already replace the entire site corpus.

7 The Rocq route

7.1 Current production files

The reviewed live files under `rocq/rocq-extension/coq/215/` are:

File	Bytes	Lines
`DomRho8.v`	22,673	1,169
`Cert215Offsets.v`	1,499,170	2,245
`Source215.v`	768	14

The facade checks `ocheck` forward and on the mirrored relation. Its generator still produces that checked route. These are measurements of live pinned files, not historical alternative sources.

7.2 Semantic contracts in the current runtime

Rocq has the same coupling. `Interpreter230Sites` defines `run_to`, `run_halt` and `check_site`. `Sim230.inv_view`, `inv_step` and `inv_halt` carry Boolean facts into the simulation and unpack them branch by branch.

Define

$$\mathrm{RunSpec}(br, Q, P, ovf, x) := \forall n, vs,\ \mathrm{valid}(br, vs) \Longrightarrow \\ \mathrm{conc}(n, vs, \mathrm{start}(Q)) \longrightarrow^{+} \mathrm{conc}(n, vs, \mathrm{tgt_conf}(P, ovf, x)).$$

The halt specification reaches a terminal configuration. `EdgeSpec` combines depth, register bounds, both decrement outcomes and the exact effect. Existing `run_to_sound` and `site_halt` supply the old-checker adapter.

Preserve the coverage index

$$\mathrm{firstn}\big(d, P +\!\!+ \mathrm{repeat}(\mathrm{false}, d)\big),$$

including implicit zero padding. The existing tape invariant and public endpoint can remain while consumers eliminate `EdgeSpec` instead of Boolean gates.

The machine-to-register direction of `Sim230Main.tm230_rm` invokes `progress_halts`, whose positive premise comes from `inv_step`. The reason for strict progress is thus identical in both systems.

The packet's `SemanticSites.proposal.v` supplies concrete definitions and proposed adapter scripts. It is uncompiled and does not yet edit the simulation consumers. It must not be cited as a new Rocq kernel result.

7.3 Share derivation data, retain independent kernel checking

A Rocq record with carrier, implication, reflexive growth relation, MP and restricted rho8 expresses the same generic theorem. Render the same 63-step data into explicit proof terms in each assistant, and connect each concrete Nat adapter independently.

The initial recommendation is direct replay. Unification and search stay untrusted; each kernel checks every explicit step. A reflective interpreter may later be useful, but its correctness proof could outweigh the benefit for a 63-step derivation.

The current 52-step row-only proposal and a shared 63-step core have different interfaces. The latter also supplies composition, prefixing and K/Peirce/S. An isolated shortest row count is not the right metric for total duplicated mathematics across the endpoint.

7.4 Existing normalized correspondence is a separate adoption

The earlier normalized Rocq package records:

Representation	Entries	Source bytes
Current offset certificate	2,240	1,499,170
Compact ordinary certificate and codec	2,240	173,003
Normalized ordinary certificate and codec	1,426	109,958

It contains `ncheck_mirror` and `ncheck_one_sided_iff`, removing the second traversal by theorem [11]. These are prior contributions.

Adopt the generator, facade and manifest together, then rebuild the affected Rocq closure, endpoint assumption reports and recursive `coqchk`. A checked theorem in an incoming package does not establish that the maintained endpoint imports it. Keep this integration separate from the semantic refactor so the acceptance question remains clear.

8 An obstruction to diagonal-only simplification

A tempting shortcut keeps only rho8 instances with $u = a$, removing the growth relation. This is invalid for the abstract interface.

Proposition 8.1. *MP and all diagonal $R(a, b, c, a)$ do not imply every unrestricted rho8 instance for arbitrary carriers and operations.*

Proof. Let $A = \{0, 1, 2\}$, designate only 0, and define

$\to$	0	1	2
0	0	1	1
1	0	0	0
2	0	0	1

MP preserves designation: with designated antecedent 0, the first row is designated only when the consequent is 0. All 27 diagonal instances are designated, by a finite case split. But $R(1, 0, 2, 2) = 1$ is not designated. □

An `Init`-only Lean theorem proves the universal failure, and an independent Python checker verifies all nine MP cases and 27 diagonal instances. This is an algebraic countermodel for an abstract propositional inference, not a model of ZFC. It rules out the diagonal-only shortcut; it does not prove independent necessity of each growth operation.

The always-true adapter is different: its unrestricted premise supplies every row. It does not weaken a restricted premise to diagonal rows. Keeping that distinction prevents an apparent interface simplification from silently changing the theorem.

9 Validation and limits

9.1 Checks performed

The compiler is Lean 4.32.0, release commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`, matching `lean-toolchain`. Proof compilers ran serially. The delivered drivers use one worker, a 12,288 MiB cap and the shared compiler-lock convention.

The checks cover:

1. The full 63-step generic core and exact replacement generic `Lukasiewicz` section. Ten named reports have no axiom dependencies.

2. The diagonal countermodel and universal obstruction. Four named reports have no axiom dependencies.
3. The 13-module simulation closure, including runtime, path, source-tree and framing dependencies.
4. Original `KSim` and a typed legacy client, then replacement `KSim` and the exact same client.
5. A semantic client for halting equivalence, prefix composition and mixed coverage.
6. Exact regeneration, preserved public headers and structure, unchanged Tarski tail, unchanged non-core renderer outputs, patch applicability and source hashes.

In the semantic audit, `semantic_halts_iff` uses only `propext` and `Quot.sound`. The legacy wrapper includes `Classical.choice` through the old provider. This local observation does not imply that the complete ZFC endpoint loses choice.

The final semantic run has 17 compiler invocations. No machine-specific site batch is re-evaluated in that focused run. Its recorded elapsed seconds are verification receipts, not an endpoint performance comparison.

9.2 What production acceptance still requires

The standalone implication projection excludes the unchanged Tarski bridge and its wider dependencies. It validates all changed mathematical proofs and generic adapters, but the full replacement `Lukasiewicz.lean` still needs project integration. The semantic checks similarly validate a generic interface, not missing primitive contracts or the entire closed K215 endpoint.

After applying the changes:

1. Regenerate the core. Compile the full `Lukasiewicz`, `DomRho8`, `KCodeDom` and affected descendants with exact source/import bindings.
2. Rebuild affected simulation descendants and rerun both closed Lean endpoint clients.
3. Retain dense-table, source, boot and assumption checks.
4. Apply separate generator and endpoint gates to any Rocq adoption; this packet has no new Rocq compiler receipt.

Proof changes invalidate downstream compiled artifacts even when executable definitions remain unchanged. Phase nine documents why valid reuse must follow source/import ancestry rather than an unchanged intermediate object [2]. Accordingly, the complete modified endpoint is not labelled checked here.

9.3 Reproduction

The packet includes exact replacements, patches, derivation and renderer, drivers, metrics and raw logs. It omits compiled objects; the drivers create isolated build directories.

```
python3 verify_all.py --repo /path/to/ProveIt --lean /path/to/lean
```

The checkout must have the pinned HEAD. Verification does not edit it and does not download tools. These focused checks use the pinned project sources and Lean's `Init`, with no Mathlib download.

Before applying:

```
git -C /path/to/ProveIt apply --check /path/to/combined.patch
```

Raw compiler logs establish compilation claims. Metrics and finite checks are recorded separately. The provenance notice retains upstream attribution.

10 Recommended integration order

Adopt the shared implication change first. It is a coherent mathematical refactor with a measured reduction, a five-step extension readable by hand, synchronized producers and a dependency-light kernel check. The numerical adapter need not be redesigned.

Next adopt semantic simulation with its compatibility wrappers and retain the existing Boolean provider. Its additional 126 lines provide an explicit mathematical boundary for composition.

Then prove one complete primitive and return family from the exact decoder port. Use mixed coverage to substitute that family, check identical branch targets and all-gap semantics, and measure clean and incremental compilation cost. Extend only after that first family demonstrates useful savings.

For Rocq, reconcile the maintained boundary with existing normalized and shorter implication proposals in separate changes. Share mathematical contracts and derivation data while each kernel checks its own proof terms. This concentrates future work on obligations that actually differ between representations.

A Source and evidence map

All repository references are pinned. `source-pins.json` records primary source hashes. Proof metrics, compiler receipts and controls are in the implementation directories and `verification/`.

Artifact	Purpose
`shared-implication/`	Four-file refactor, derivation, renderer, standalone projection, replay driver and metrics.
`semantic-sites/`	Replacement `KSim`, driver and old/new typed clients.
`countermodel/`	Three-element algebra, Lean obstruction and Python check.
`rocq-design/`	Explicitly uncompiled interface draft and integration notes.
`verification/core/`	Core/countermodel logs, hashes and source controls.
`verification/sites/`	Semantic receipt and original/replacement client logs.
`combined.patch`	Independent implication and simulation changes together.
`verify_all.py`	Serial focused verification against the pinned checkout.
`NOTICE.txt`	Attribution and boundary between new and inherited work.

References

[1] ProveIt, *Formal correctness of the exact 215-state machine*. Verification record, with exact hashes, endpoint statements and internal-proof boundary.

[2] ProveIt, *Ninth K215 proof-simplification checkpoint*. Phase-nine record, including Home, decoder prefix, retained outcomes and timing qualifications.

[3] ProveIt, `DominatedImplication.lean`, the maintained 58-step core.

[4] ProveIt, `Lukasiewicz.lean`, the 107-helper derivation and Tarski bridge.

[5] ProveIt, `rho8cd/PROVEIT_STATUS.md`, distinguishing maintained Lean and Rocq boundaries.

[6] ProveIt, `KSim.lean`, original checker, coverage and simulation.

[7] ProveIt, `KDecoderPrefix215.lean`, exact zipper entry and uniform leaf/walk prefixes.

[8] ProveIt, *Context factoring and a universal return-to-dispatch contract*. Earlier runtime-context proposal, including the semantic site-specification design.

[9] ProveIt, Context-aware decoder-sharing research, local frames and limitations of state/read-bit sharing.

[10] ProveIt, Rocq `Source215.v` and `DomRho8.v`, plus the matching generator.

[11] ProveIt, Normalized Rocq correspondence proposal, with 1,426 entries and the one-sided equivalence theorem.

[12] ProveIt, Core derivation and renderer. The packet supplies synchronized successors.